🧠 1. PAGING

🔷 1. INTRODUCTION (Why Paging exists)

Operating System ko problem hoti hai:

- Process continuous memory demand karta hai
- Continuous memory allocate karna hard hota hai
- External fragmentation problem hoti hai

👉 Solution: **Paging**

🔷 2. DEFINITION (Technical)

Paging is a memory management technique in which a process is divided into fixed-size pages and physical memory is divided into fixed-size frames, allowing non-contiguous allocation and efficient utilization of memory.

🔷 3. BASIC CONCEPT

- Logical memory = Pages
- Physical memory = Frames
- Page size = Frame size (same)

👉 Process ko chhote blocks me tod diya jata hai

🔷 4. ARCHITECTURE (HOW IT WORKS)

Step-by-step flow:

1. CPU logical address generate karta hai
2. Logical address = Page Number + Offset
3. Page Table lookup hota hai
4. Frame number milta hai
5. Physical address banata hai
6. RAM se data fetch hota hai

🔷 5. ADDRESS TRANSLATION (VERY IMPORTANT)

Logical Address:

[Page Number | Offset]

Physical Address:

[Frame Number | Offset]

👉 Page Table mapping karta hai:
Page No → Frame No

🔷 6. PAGE TABLE (CORE COMPONENT)

Page Table contains:

- Page Number
- Frame Number
- Valid/Invalid bit
- Protection bits

👉 Stored in RAM

🔷 7. HARDWARE SUPPORT (MMU)

MMU (Memory Management Unit):

- CPU aur RAM ke beech hota hai
- Logical → Physical conversion karta hai

🔷 8. TLB (TRANSLATION LOOKASIDE BUFFER)

👉 Fast cache for page table

- Stores recent mappings
- Speeds up address translation

🔷 9. FRAGMENTATION

Internal Fragmentation:

- Last page me unused space

External Fragmentation:

❌ Paging me nahi hota

🔷 10. PAGE FAULT (IMPORTANT)

Jab required page RAM me nahi hota:

Steps:

1. CPU interrupt generate karta hai
2. OS page disk se load karta hai
3. Free frame assign hota hai
4. Page RAM me load hota hai

🔷 11. PAGE REPLACEMENT (ADVANCED)

Jab RAM full hoti hai:

Algorithms:

- FIFO
- LRU
- Optimal

🔷 12. ADVANTAGES

- No external fragmentation
- Easy memory allocation
- Efficient utilization
- Multiprogramming support

🔷 13. DISADVANTAGES

- Internal fragmentation
- Page table overhead
- Extra memory access time

🔷 14. REAL-LIFE ANALOGY

Library books:

- Pages = chapters
- Frames = shelves
- Page table = index card

🔷 15. INTERVIEW DEFINITION (FINAL)

Paging is a non-contiguous memory management technique where processes are divided into fixed-size pages and mapped to physical memory frames using a page table managed by MMU, eliminating external fragmentation.

🧠 2. SEGMENTATION (TOP TO BOTTOM DEEP STUDY)

🔷 1. INTRODUCTION

Paging memory ko fixed size me todta hai

👉 But problem: logical structure missing

👉 Solution: **Segmentation**

🔷 2. DEFINITION (Technical)

Segmentation is a memory management technique in which a process is divided into variable-sized logical segments such as code, data, stack, and heap, and each segment is independently mapped into memory.

🔷 3. BASIC CONCEPT

Process is divided into logical parts:

- Code segment
- Data segment
- Stack segment
- Heap segment

🔷 4. ADDRESS STRUCTURE

Logical Address:

[Segment Number | Offset]

Segment Table:

- Base Address
- Limit

🔷 5. ADDRESS TRANSLATION

Steps:

1. CPU generates segment number
2. Segment table lookup
3. Base + offset calculation
4. Physical memory access

🔷 6. BOUND CHECKING

If:

Offset > Limit → ERROR (Segmentation Fault)

🔷 7. FRAGMENTATION

External Fragmentation:

✓ YES (major issue)

Internal Fragmentation:

❌ No (variable size allocation)

🔷 8. ADVANTAGES

- Logical view of program
- Better security
- Easy sharing of code/data

🔷 9. DISADVANTAGES

- External fragmentation
- Memory compaction needed
- Complex allocation

🔷 10. REAL-LIFE ANALOGY

Building:

- Floors = segments

- Each floor different size

🔷 11. INTERVIEW DEFINITION

Segmentation is a memory management technique where a process is divided into variable-sized logical units and mapped into memory using segment tables providing logical program structure and protection.

🧠 3. VIRTUAL MEMORY (TOP TO BOTTOM DEEP STUDY)

🔷 1. INTRODUCTION

Problem:

- RAM limited
- Programs large

👉 Solution: Virtual Memory

🔷 2. DEFINITION (Technical)

Virtual memory is a memory management technique that allows execution of processes larger than physical memory by using secondary storage (disk) as an extension of RAM.

🔷 3. BASIC IDEA

- Only required part loaded in RAM
- Remaining stored in disk
- On-demand loading

🔷 4. KEY CONCEPTS

1. Demand Paging

Pages tab load hote hain jab required ho

2. Page Fault

Page RAM me nahi milta

3. Swap Space

Disk area used as backup memory

🔷 5. PAGE FAULT HANDLING

Step-by-step:

1. CPU access page
2. Page not found → page fault
3. OS interrupt
4. Disk se page load
5. Frame assign
6. Resume execution

🔷 6. THRASHING

👉 When too many page faults occur

Effect:

- System slow
- CPU time wasted in swapping

🔷 7. ADVANTAGES

- Large programs run possible
- Efficient RAM usage
- Multiprogramming increased

🔷 8. DISADVANTAGES

- Slower execution
- Disk overhead
- Thrashing risk

🔷 9. REAL-LIFE ANALOGY

Desk (RAM) + cupboard (disk):

- Only needed files on desk
- Rest stored in cupboard

🔷 10. INTERVIEW DEFINITION

Virtual memory is a technique that extends physical memory by using secondary storage, allowing execution of large processes through demand paging and efficient memory abstraction.

🧠 4. FILE SYSTEM (TOP TO BOTTOM DEEP STUDY)

🔷 1. INTRODUCTION

Computer ko data store karna hota hai structured way me

👉 Solution: File System

🔷 2. DEFINITION (Technical)

A file system is a method used by an operating system to organize, store, retrieve, and manage data on storage devices in the form of files and directories.

🔷 3. FILE CONCEPT

File = collection of related data

Example:

- text file
- image file
- program file

🔷 4. FILE STRUCTURE

Components:

- Data
- Metadata (size, type, permissions)

🔷 5. FILE OPERATIONS

- Create
- Open
- Read

- Write
- Close
- Delete

🔷 6. FILE ACCESS METHODS

1. Sequential Access

- One by one read

2. Direct Access

- Random access possible

3. Indexed Access

- Index table used

🔷 7. FILE ALLOCATION METHODS

1. Contiguous Allocation

- Continuous blocks

2. Linked Allocation

- Pointer-based chaining

3. Indexed Allocation

- Index block used

🔷 8. DIRECTORY STRUCTURE

- Single level
- Two level
- Tree structure
- Graph structure

🔷 9. FILE SYSTEM TYPES

- FAT
- NTFS
- EXT (Linux)

🔷 10. ADVANTAGES

- Organized storage
- Easy access
- Security support

🔷 11. DISADVANTAGES

- Fragmentation
- Complexity
- Overhead

🔷 12. INTERVIEW DEFINITION

File system is an OS component that manages how data is stored, organized, accessed, and maintained on storage devices using files and directories.

🚀 FINAL SUMMARY (VERY IMPORTANT)

Topic	Core Idea
Paging	fixed-size memory mapping
Segmentation	logical variable-size memory
Virtual Memory	RAM + disk extension
File System	structured data storage system